

BRDGIT

Ads Automation Pipeline

Complete Reference — Architecture, Skills, Templates, Hardening Plan

Stack

Claude Code + Figma MCP + Google Ads MCP + Google Sheets
MCP

Terminal-native. No external orchestrators. No middleware.

Document version 1.1 — includes hardening plan addressing 13 architectural risks

Table of Contents

- 1. Executive Overview
- 2. System Architecture
- 3. Google Sheet Schema (the campaign brief)
- 4. Campaign Naming Convention
- 5. Figma Design System Setup
- 6. The 4 Approval Gates (in detail)
- 7. Pipeline Flow Diagram
- 8. Budget & Safety Controls
- 9. All Skills (13 modules, full reference)
- 10. Prompt Templates (10 ready-to-use)
- 11. Day-to-Day Operations
- 12. Files & Project Structure
- 13. Architectural Risks Identified (v1.0)
- 14. Hardening Plan (v1.1) — concrete improvements
- 15. Phase 2 Roadmap — AI-Powered Optimization

1. Executive Overview

An end-to-end campaign automation pipeline built on Claude Code. It takes a campaign brief from a Google Sheet and turns it into deployed (paused) campaigns in Google Ads — with Figma creatives — without any manual setup work.

Core principle: Claude Code is the brain. You are the approver.

Every step is transparent. Nothing goes live without your explicit "approved" or "deploy" at each gate. The pipeline handles the mechanical work — reading data, generating copy, creating designs, calling APIs — and surfaces decisions to you in a clean, reviewable format.

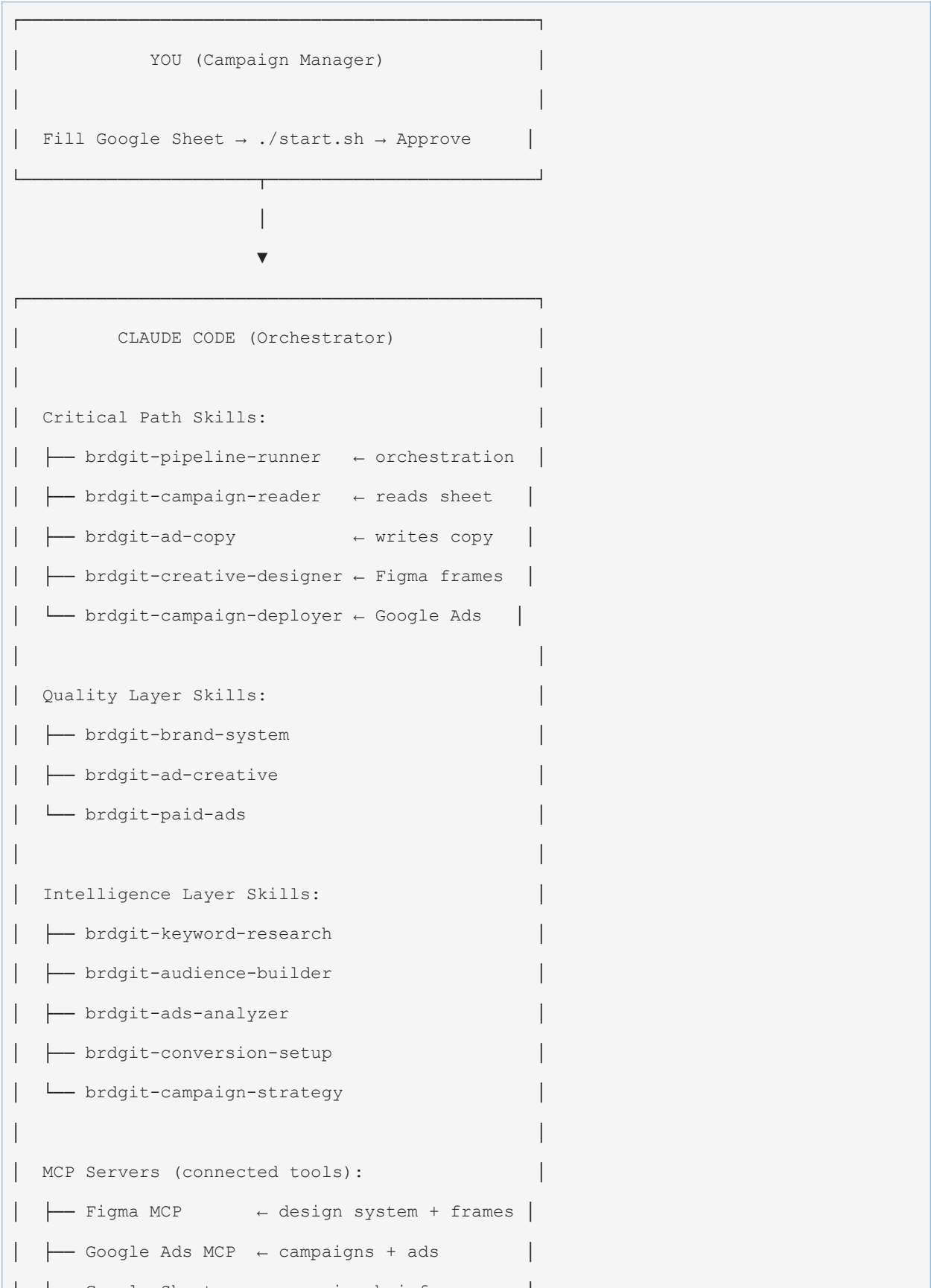
What you get

- Campaign briefs in a single Google Sheet, deployed via one command
- AI-generated headlines, descriptions, and ad extensions (with human review)
- Figma creatives generated automatically, brand-aligned, ready for upload
- Google Ads campaigns, ad groups, keywords, and ads created via API
- All campaigns deployed PAUSED — you flip the switch manually
- Every step logged and reviewable

What this is NOT

- Not a SaaS — it runs on your machine, with your accounts, using your credentials
- Not autopilot — every gate requires your explicit approval
- Not a copy generator wrapper — it integrates copy + creative + deployment in one flow

2. System Architecture



3. Google Sheet Schema

The pipeline reads from a single Google Sheet with exactly 5 tabs. This sheet is the source of truth for every campaign. Row 1 = headers, data starts at Row 2.

SHEET NAME

BRDGIT Campaign Brief — [Quarter] (e.g., "BRDGIT Campaign Brief — Q3 2026")

Tab 1 — campaign_settings

Defines every campaign at the campaign level. One row per campaign.

Column	Type	Required	Validation
campaign_name	string	✓	Must follow naming convention
campaign_type	enum	✓	search, display, pmax, video, shopping
objective	enum	✓	conversions, traffic, awareness, app_installs
daily_budget	number	✓	> 0, max \$500 (pipeline cap)
bid_strategy	enum	✓	maximize_conversions, target_cpa, target_roas, manual_cpc, maximize_clicks
target_cpa	number	if target_cpa	USD value
target_roas	number	if target_roas	percentage (e.g., 400 = 400%)
start_date	date	✓	YYYY-MM-DD, today or future
end_date	date	optional	must be after start_date
geo_targets	string	✓	Comma-separated: "US" or "US, CA, UK"
language	string	✓	ISO 639-1: "en", "pt", etc.
network	enum	for search	search_only, search_and_display
deploy_status	enum	✓	Always "paused" — pipeline enforces

Tab 2 — ad_groups

Defines ad groups. Many rows per campaign.

Column	Type	Required	Notes
campaign_name	string (FK)	✓	Must match a name in campaign_settings
ad_group_name	string	✓	
keywords	string	✓ for search	Comma-separated list
match_type	enum	✓	broad, phrase, exact
negative_keywords	string	optional	Comma-separated
cpc_bid_override	number	optional	USD — overrides campaign-level bid

Tab 3 — ad_copy

Defines headlines, descriptions, and final URLs. Up to 15 headlines and 4 descriptions per ad group.

Column	Type	Required	Constraint
campaign_name	string (FK)	✓	
ad_group_name	string (FK)	✓	
headline_1	string	✓ (min 3)	max 30 chars
headline_2	string	✓	max 30 chars
headline_3	string	✓	max 30 chars
headline_4 to headline_15	string	optional	max 30 chars each
description_1	string	✓ (min 2)	max 90 chars
description_2	string	✓	max 90 chars
description_3 to description_4	string	optional	max 90 chars each
final_url	URL	✓	valid URL
display_path_1	string	optional	max 15 chars
display_path_2	string	optional	max 15 chars
pin_headline_1_to	number	optional	1, 2, or 3

TIP

Leave headline_1 as "GENERATE" to trigger AI copy generation for that ad group.

Tab 4 — extensions

Column	Type	Required	Notes
campaign_name	string (FK)	✓	
extension_type	enum	✓	sitelink, callout, structured_snippet, call
text	string	sitelinks, callouts	
url	URL	sitelinks	
description_1	string	sitelinks	max 35 chars
description_2	string	sitelinks	max 35 chars
header	string	structured_snippets	
values	string	structured_snippets	Comma-separated

Tab 5 — creative_specs

Column	Type	Required	Notes
campaign_name	string (FK)	✓	
ad_group_name	string (FK)	optional	
creative_type	enum	✓	banner, square, portrait, logo_lockup, medium_rectangle
width	number	✓	pixels
height	number	✓	pixels
headline_text	string	✓	text on the creative
body_text	string	optional	secondary text
cta_text	string	✓	button text
color_scheme	string	optional	brand_primary, brand_dark, brand_light, hex
image_direction	string	optional	art direction notes
figma_frame_name	string	✓	unique name for the Figma frame

4. Campaign Naming Convention

Pattern: BRDGIT_[CampaignType]_[Audience]_[Offer]_[Quarter]

Campaign Types

Code	Platform / Type
Search	Google Ads Search
PMAX	Google Performance Max
Display	Google Display
Video	Google Video (YouTube)
LinkedIn	LinkedIn Ads
Meta	Meta Ads (Facebook + Instagram)

Audience Codes

Code	Description
B2B	General B2B / all decision-makers
CTO	Chief Technology Officers
VPEng	VP of Engineering / Head of Engineering
Founders	Founders / CEOs with technical background
HeadOfProduct	Head of Product / VP Product
AllAudiences	No specific persona (PMAX, broad)

Offer Codes

Code	Description
FractionalAI	Core service offer
DemoBooking	Book a demo / call
FreeTrial	Free trial or audit

Examples

```
BRDGIT_Search_B2B_FractionalAI_Q3  
BRDGIT_Search_CTO_DemoBooking_Q3-2026  
BRDGIT_PMAX_AllAudiences_FractionalAI_Q3  
BRDGIT_LinkedIn_Founders_DemoBooking_Q3  
BRDGIT_Meta_VPEng_FractionalAI_Q3
```

Ad Group & Figma Frame naming

Ad groups: [Keyword Theme] — [Match Type] (e.g., "Fractional AI — Phrase")

Figma frames: BRDGIT_[Quarter]_[AdGroup]_[Width]x[Height] (e.g., "BRDGIT_Q3_FractionalAI_1200x628")

5. Figma Design System Setup

Set this up once — then the pipeline can generate creatives autonomously.

Required File Structure

```
Your Figma File
├─ Page: Design System
|   ├─ Color Variables
|   ├─ Text Styles
|   └─ Component Library
|       ├─ Buttons/   (Primary, Secondary, Ghost)
|       ├─ Text/      (Display, H1, H2, Body, CTA)
|       └─ Logo/      (Full color, white, icon)
|
└─ Page: Generated Creatives - [Quarter] [Year]
    └─ ← pipeline creates frames here
```

Color Variables

Create these variables in a collection named "BRDGIT Brand":

Variable	Hex
color/primary	#2E75B6
color/dark	#1A1A2E
color/accent	#FF6B35
color/light	#F5F5F5
color/white	#FFFFFF

Text Styles

Style	Font	Weight	Size	Use
Display/Hero	Sentient	Bold	48–80px	Hero headlines on banners
Headline/H1	Sentient	Bold	32–48px	Main headlines
Headline/H2	Satoshi	Bold	24–32px	Secondary headlines
Body/Large	Satoshi	Regular	18–20px	Body on large formats
Body/Small	Satoshi	Regular	14–16px	Body on 300×250
CTA/Label	Satoshi	Bold	16–18px	Button text

Ad Template Frames (pre-built components)

Template	Dimensions	Notes
Ad/Banner/1200x628	1200×628	Standard horizontal banner
Ad/Square/1080x1080	1080×1080	Instagram square
Ad/MediumRect/300x250	300×250	Google Display medium rectangle
Ad/Portrait/628x1200	628×1200	Stories / vertical

Each template should have: background layer (color variable), headline (Sentient Bold), body (Satoshi Regular, optional), CTA button component, logo, safe-zone guides 40px from edges.

6. The 4 Approval Gates

The pipeline never moves to the next step without your explicit approval. Every gate shows you exactly what will happen next.

GATE 1: Plan Review

Triggered immediately after reading the spreadsheet

GATE 1: PLAN REVIEW

Campaign: BRDGIT_Search_B2B_FractionalAI_Q3

Type: Search | Objective: Conversions

Budget: \$75/day (\$2,280/mo) | Bid: Target CPA @ \$18

Geo: US, CA, UK | Language: EN

Ad Groups:

- "Fractional AI" — 5 keywords (phrase) + 3 negatives
- "Brand Terms" — 3 keywords (exact)

Creatives: 1× banner (1200×628), 1× square (1080×1080)

⚠ No warnings. Budget within safety limits.

Approve? (approved / change X to Y / abort)

Automatic safety checks at Gate 1:

- Validates all required fields in the spreadsheet
- Confirms no campaign budget exceeds \$500/day (hard cap)
- Warns if total monthly projection exceeds \$10,000
- Flags any date in the past
- Overrides `deploy_status` to "paused" if anything else was entered

GATE 2: Copy Review

Triggered after plan is approved

Three paths:

- Path A — Copy in spreadsheet: Claude validates and displays for review
- Path B — "GENERATE" in ad_copy: Claude writes copy using the 6 angles (Pain Point, Outcome, Social Proof, Curiosity, Comparison, Identity)
- Path C — Hybrid: you wrote some, Claude fills the rest. Each marked  you /  generated

GATE 2: COPY REVIEW – "Fractional AI"

HEADLINES (8 of 15 max — 30 char limit):

#	Headline	Chars	Source
1	AI Projects Keep Stalling?	26	you
2	Ship AI Projects Faster	22	you
3	Senior AI Engineers Ready	25	gen
4	CTOs Choose BRDGIT	19	gen

ESTIMATED AD STRENGTH: Good

Approve? (approved / regenerate headline 3 / abort)

GATE 3: Creative Review

Triggered after copy is approved

Claude creates all Figma frames specified in `creative_specs` — applying brand colors, typography (Sentient for headlines, Satoshi for body), and CTA buttons from the design system.

GATE 3: CREATIVE REVIEW

Created 2 frames in Figma:

#	Frame Name	Size	✓
1	BRDGIT_Q3_FractionalAI_1200x628	1200x628	✓
2	BRDGIT_Q3_FractionalAI_1080x1080	1080x1080	✓

Review link: <https://figma.com/file/...>

Approve? (approved / redo frame 1 / abort)

GATE 4: Deploy Confirmation

Triggered after creatives are approved — runs a dry run first

GATE 4: DEPLOYMENT SUMMARY — DRY RUN

CAMPAIGNS TO CREATE: 1
AD GROUPS TO CREATE: 2
ADS TO CREATE: 2
KEYWORDS TO ADD: 8 (+ 3 negatives)
EXTENSIONS: 2 sitelinks, 4 callouts

BUDGET IMPACT:

Daily spend (max):	\$75.00
Monthly projection:	\$2,280
Bid strategy:	Target CPA
Initial status:	PAUSED ✓

Type "deploy" to execute or "abort" to cancel.

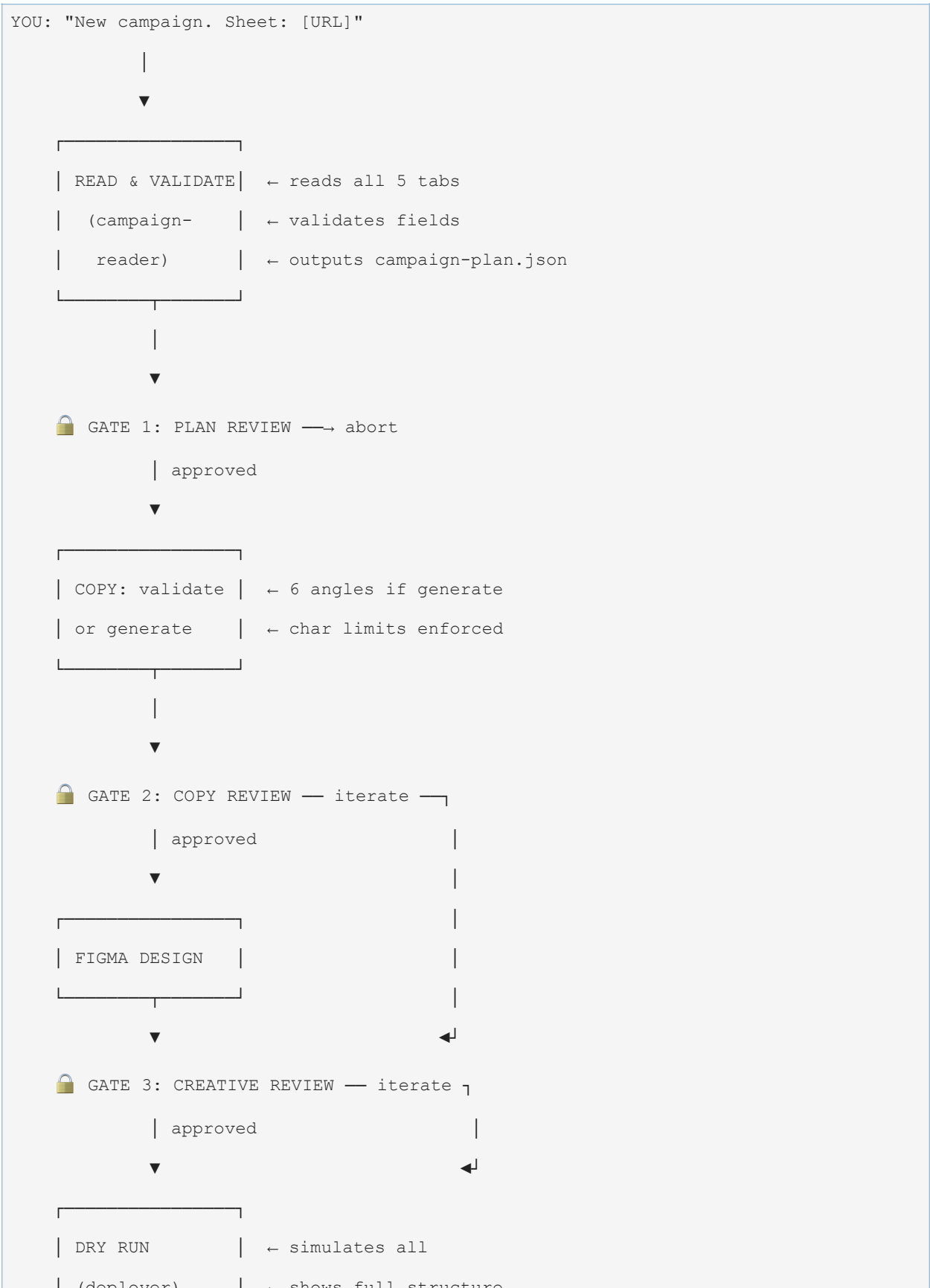
After "deploy" — execution sequence:

- Campaign budget
- Campaign (status: PAUSED)
- Geo and language targeting criteria
- Ad groups (×N)
- Keywords with match types (×N)
- Negative keywords
- Responsive Search Ads with all headlines and descriptions
- Sitelink and callout extensions
- Image assets uploaded (if PMAX)
- Verification call → confirms structure was created correctly

YOU GO LIVE MANUALLY

The pipeline never enables campaigns automatically. You review everything in Google Ads, confirm it looks right, then flip the switch yourself.

7. Pipeline Flow Diagram



8. Budget & Safety Controls

These rules are hard-coded into the deployer skill and cannot be overridden by a prompt:

Rule	Behavior
Daily cap	Refuses any campaign with daily budget > \$500. To raise: edit the skill file.
Monthly warning	If total monthly spend > \$10,000, requires explicit confirmation: "I confirm the total monthly budget of \$X".
Always paused	All campaigns deploy with status = PAUSED. No exceptions.
Dry run first	The deployer always shows a dry run before executing live API calls.
Budget change alert	On updates, highlights the delta and requires confirmation.
Stop on error	Pipeline stops on first failure, logs created resources, reports the exact error.

9. All Skills (13 modules, full reference)

All skills live in `.claude/skills/` and load automatically when you open Claude Code from the project directory.

Tier 1 — Critical Path (pipeline depends on these)

brdgit-pipeline-runner

File: `.claude/skills/brdgit-pipeline-runner.md`

Purpose: Master orchestration. Chains all steps end-to-end, enforces all 4 gates, formats gate displays, handles batch mode (multiple campaigns from one sheet).

Triggers:

- "New campaign. Sheet: [URL]"
- "Run the full campaign pipeline"
- "Batch deploy all campaigns from Sheet: [URL]"

Output: Saves `/reports/deploy-[date]-[campaign].md` with all created IDs and dashboard links.

brdgit-campaign-reader

File: `.claude/skills/brdgit-campaign-reader.md`

Purpose: Reads the Google Sheet via Sheets MCP. Validates all 5 tabs, cross-references foreign keys (campaign_name across tabs), flags missing data, and emits a normalized `campaign-plan.json`. Hard rule: never guesses missing fields, always overrides `deploy_status` to "paused".

Triggers:

- Used internally by pipeline-runner
- Manual: "Read campaign brief from [URL] and validate"

Output: `campaign-plan.json` with all campaigns, ad groups, copy, extensions, creative specs.

brdgit-campaign-deployer

File: `.claude/skills/brdgit-campaign-deployer.md`

Purpose: Executes the full Google Ads API sequence: budget → campaign → geo → ad groups → keywords → negatives → RSA → extensions → assets → verification. Enforces the budget cap, the always-paused rule, and dry-run-first. Logs every resource ID for rollback / audit.

Triggers:

- Used internally by pipeline-runner
- Manual: "Deploy campaign-plan.json to account [ID] in dry-run mode"

Output: Deployment report in /reports/ with dashboard links and resource IDs.

brdgit-creative-designer

File: .claude/skills/brdgit-creative-designer.md

Purpose: Generates Figma frames via Figma MCP. Reads brand variables (colors, typography), duplicates pre-built ad templates, fills in copy, places logo, exports PNG assets for PMAX uploads. Names frames according to convention.

Triggers:

- Used internally by pipeline-runner
- Manual: "Generate creatives for [campaign_name] in Figma [URL]"

Output: figma-assets.json with frame names, Figma URLs, and exported PNG paths.

Tier 2 — Quality Layer (make outputs better)

brdgit-brand-system

File: .claude/skills/brdgit-brand-system.md

Purpose: Single source of truth for brand identity: primary #2E75B6, dark #1A1A2E, accent #FF6B35; Sentient (headlines) and Satoshi (body); ICP (CTOs/VPs Eng/Founders at 50–500-employee companies); approved CTAs; banned words; tone of voice (professional, direct, no hype).

Triggers:

- Auto-loaded by ad-copy and creative-designer
- Manual: "What CTAs are approved for BRDGIT?"

Output: Reference document — referenced by every other content-generating skill.

brdgit-ad-copy

File: .claude/skills/brdgit-ad-copy.md

Purpose: Writes and iterates Google RSA copy. Applies the 6 copy angles (Pain Point, Outcome, Social Proof, Curiosity, Comparison, Identity). Enforces 30/90 char limits.

Estimates Ad Strength. Displays copy in a table with character counts and 📏🤖 source markers.

Triggers:

- "Generate ad copy for [ad group]"
- "Regenerate headline N — focus on speed"

- "Add 3 more headlines about [angle]"

Output: Updated ad_copy section in campaign-plan.json with strength estimate.

brdgit-ad-creative

File: .claude/skills/brdgit-ad-creative.md

Purpose: Generates complete creative packages across formats: Google RSA, Meta (5 angles, 125–400 char primary text), LinkedIn (5 personas), PMAX asset checklist (15 headlines, 5 descriptions, 5 long headlines, images, videos, logos).

Triggers:

- "Generate complete creative package for [campaign]"
- "Adapt this campaign for Meta and LinkedIn"

Output: Multi-platform creative document, ready for Figma and ad upload.

brdgit-paid-ads

File: .claude/skills/brdgit-paid-ads.md

Purpose: Campaign architecture defaults: bid strategies by funnel phase (Maximize Clicks for cold → Target CPA for warm → Target ROAS for hot), recommended ad-group structure, budget allocation across phases, match-type strategy (Exact + Phrase, no Broad), default negative keyword lists.

Triggers:

- "What's the recommended structure for a Q4 cold-traffic search campaign?"

Output: Campaign architecture recommendation.

Tier 3 — Intelligence Layer (research & analysis)

brdgit-keyword-research

File: .claude/skills/brdgit-keyword-research.md

Purpose: Pre-loaded with 30 researched keywords (US market) across Awareness, Consideration, and Decision stages, with estimated monthly volumes, competition difficulty, and intent type. Match-type strategy + negative keyword lists. Expansion process for new themes.

Triggers:

- "Research keywords for ad group: [theme]"
- "Suggest negatives for [campaign]"

Output: Keyword list with match types, volumes, intents, and negatives.

brdgit-audience-builder

File: .claude/skills/brdgit-audience-builder.md

Purpose: Defines audience segments across Google Ads (custom segments, in-market, 5 remarketing lists, customer match), LinkedIn (4 segments by title/seniority/company size), and Meta (cold + retargeting audiences).

Triggers:

- "Build audience for LinkedIn campaign targeting VPs of Engineering"
- "Set up remarketing for visitors who saw /pricing"

Output: Audience definitions with platform-specific configuration.

brdgit-ads-analyzer

File: .claude/skills/brdgit-ads-analyzer.md

Purpose: Pulls performance data via Google Ads MCP. Evaluates against KPI thresholds (CTR > 3%, CPC < \$8, CPL < \$150). Surfaces underperformers, winning patterns, and search terms to add/exclude. Generates weekly reports with optimization playbooks.

Triggers:

- "Analyze campaign performance for last 14 days"
- "What's the worst-performing keyword this month?"

Output: Performance report with // flags and recommended actions.

brdgit-conversion-setup

File: .claude/skills/brdgit-conversion-setup.md

Purpose: Conversion action configuration (Demo Booked \$150 primary, Form Submit \$100, secondary conversions for engagement). GTM tag list. UTM schema: utm_source=[platform]&utm_medium=cpc&utm_campaign=[name]&utm_content=[ad_group]&utm_term={keyword}.

Triggers:

- "Set up conversion tracking for the demo booking flow"
- "What UTMs should I use for LinkedIn?"

Output: Conversion + GTM + UTM specification.

brdgit-campaign-strategy

File: .claude/skills/brdgit-campaign-strategy.md

Purpose: Quarterly planning framework. Full funnel (Cold/Warm/Hot), budget allocation by phase, audience tier definitions, campaign selection logic, 6-week sprint structure, seasonal calendar.

Triggers:

- "Plan the Q4 campaign strategy and budget allocation"
- "Should we launch a PMAX or stay Search-only?"

Output: Quarterly campaign plan with budgets and rationale.

10. Prompt Templates

Copy/paste-ready triggers for the most common operations.

1. Full Pipeline (minimal)

```
New campaign.  
  
Sheet: [GOOGLE_SHEET_URL]
```

2. Full Pipeline (explicit)

```
New campaign.  
  
Sheet: [GOOGLE_SHEET_URL]  
  
Figma: [FIGMA_FILE_URL]  
  
Account: [GOOGLE_ADS_CUSTOMER_ID]  
  
Mode: dry-run
```

3. Full Pipeline with Overrides

```
New campaign from sheet [GOOGLE_SHEET_URL].  
  
Override daily budget to ${AMOUNT}.  
  
Creative direction: [NOTES - e.g. darker scheme, more aggressive CTAs].  
  
Figma: [FIGMA_FILE_URL]  
  
Mode: dry-run
```

4. Batch Deploy (multiple campaigns)

```
Batch deploy all campaigns from Sheet: [GOOGLE_SHEET_URL]  
  
Account: [CUSTOMER_ID]  
  
Mode: dry-run first, then deploy approved campaigns
```

5. Copy Generation Only

```
Generate ad copy for:  
  
Campaign: [CAMPAIGN_NAME]  
  
Ad Group: [AD_GROUP_NAME]  
  
Audience: [CTO / VP Engineering / Founder / Head of Product]  
  
USP focus: [pain point / outcome / social proof / curiosity / comparison / identity]  
  
Format: Google RSA (15 headlines, 4 descriptions)
```

6. Creative Only (Figma)

```
Generate creatives in Figma [FIGMA_FILE_URL]:  
  
Campaign: [CAMPAIGN_NAME]  
  
Sizes: 1200x628, 1080x1080, 300x250  
  
Headline: [TEXT]  
  
CTA: [TEXT]  
  
Style: [brand_primary / brand_dark / brand_light]
```

7. Campaign Update

```
Update campaign [CAMPAIGN_NAME] from Sheet: [GOOGLE_SHEET_URL]  
  
Show me what changed before applying anything.
```

8. Copy Refresh Only

```
Regenerate copy for campaign [CAMPAIGN_NAME].  
  
Focus on: [urgency / social proof / speed / ROI]  
  
Keep headlines: [N, N, N] – regenerate the rest.
```

9. Performance Analysis

```
Analyze Google Ads performance for account [CUSTOMER_ID].  
  
Date range: last 14 days  
  
Focus on: [campaign name OR "all campaigns"]  
  
Highlight: underperformers and search terms to add/exclude.
```

10. Keyword Research

Research keywords for ad group: [AD_GROUP_NAME]

Context: [brief description]

Output: keyword list with match types and negative recommendations.

11. Day-to-Day Operations

Launch a new campaign

1. Fill the Google Sheet (use the schema in section 3)
2. `cd ~/Documents/claude-workspace/brdgit-ads-automation`
3. `./start.sh`
4. Type: "New campaign. Sheet: [URL]"
5. Approve at each gate
6. Open Google Ads → enable when ready

Run a weekly performance check

```
./start.sh  
  
"Analyze Google Ads performance for account [CUSTOMER_ID]. Last 7 days."
```

Pause a keyword

```
"Pause keyword '[exact text]' in campaign [name]. Show me what changes before  
applying."
```

Refresh creatives at quarter rollover

```
"Generate fresh creatives in Figma for the [campaign_name] campaigns.  
  
New theme: [direction]. Keep brand system intact."
```

12. Files & Project Structure

```

brdgit-ads-automation/
|
├─ .claude/
|   └─ settings.json          ← MCP server config
|   └─ skills/
|       └─ brdgit-pipeline-runner.md
|       └─ brdgit-campaign-reader.md
|       └─ brdgit-campaign-deployer.md
|       └─ brdgit-creative-designer.md
|       └─ brdgit-brand-system.md
|       └─ brdgit-ad-copy.md
|       └─ brdgit-ad-creative.md
|       └─ brdgit-paid-ads.md
|       └─ brdgit-keyword-research.md
|       └─ brdgit-audience-builder.md
|       └─ brdgit-ads-analyzer.md
|       └─ brdgit-conversion-setup.md
|       └─ brdgit-campaign-strategy.md
|
├─ config/
|   └─ pipeline.yaml          ← defaults (account, Figma, caps)
|   └─ brand-guidelines.json  ← colors, fonts, CTAs, banned words
|
├─ templates/
|   └─ spreadsheet-schema.md   ← columns + validation rules
|   └─ campaign-naming.md      ← naming convention
|   └─ figma-design-system.md  ← Figma file setup
|
├─ prompts/
|   └─ templates.md           ← 10 ready-to-use prompts
|
├─ scripts/
|
└─

```


13. Architectural Risks Identified (v1.0)

Honest assessment of where the v1.0 pipeline can fail. Each risk is addressed in section 14 (the Hardening Plan).

READ FIRST

These risks are real and have happened in similar Claude-Code-orchestrated systems. They do not block the pipeline from working today, but each one is a future incident waiting to happen if not addressed.

Critical Risks

#	Risk	Failure Mode
1	Silent MCP crash mid-deploy	Budget created in Google Ads, campaign not. No rollback.
2	OAuth refresh token revocation	Pipeline fails with invalid_grant during deploy. No pre-flight check.
3	Spreadsheet as single point of failure	Renamed tab or merged cell → reader silently reads wrong values.

Architectural Weaknesses

#	Risk	Failure Mode
4	LLM-driven orchestration = no determinism	Same input may produce different bid calculations or API call ordering.
5	No state persistence between gates	Session crash between Gate 2 and Gate 3 → restart from scratch.
6	No audit trail of human approvals	No signed log of what state was approved vs what was deployed.
7	Figma MCP is the most fragile dependency	Browser-OAuth timeout blocks the whole campaign.

Operational Gaps

#	Risk	Failure Mode
8	No conflict detection	Duplicate campaign names get created silently.
9	Budget caps editable by anyone with file access	No checksum, no separate unlock; soft enforcement.
10	Dry run isn't a real dry run	Only simulates Claude reasoning; misses real Google Ads policy/keyword errors.
11	No rate limit handling	Bulk batch deploys can hit API quotas mid-sequence and fail.

Phase 2 Risks (forecasted)

#	Risk	Failure Mode
12	Cross-industry copy adapter	Hallucinated compliance claims (HIPAA, SOC2, GDPR) — legal risk.
13	Performance-triggered auto-pause	Pauses keywords that just need more data, not more performance.

14. Hardening Plan (v1.1)

Concrete fixes for each of the 13 risks above. Each item maps 1:1.

PRIORITY P0 — DO FIRST

Items #5 (state persistence), #10 (real dry run), #2 (token pre-flight). These three give the biggest reliability gains for the lowest implementation cost.

Critical Fixes

Fix #1 — Transaction log + auto-rollback in deployer

Problem: MCP crash leaves orphan resources in Google Ads.

Solution:

- After every API call in brdgkit-campaign-deployer, append the resource ID to /reports/.in-flight-[campaign].jsonl
- On any error mid-sequence: read .in-flight log, call Google Ads API to delete every resource created during this run, in reverse order
- On success: rename .in-flight log to /reports/deploy-[date]-[campaign].md

Impact: A failed deploy ends in a clean Google Ads account, not a half-built campaign.

Fix #2 — Pre-flight token check in start.sh

Problem: OAuth refresh tokens silently revoke after 6 months of inactivity, password changes, or 7+ apps using the account.

Solution:

- In start.sh, after sourcing .env, run a small script that hits the Google Ads API with `customer.list` and the Sheets API with a no-op read
- If either returns 401/invalid_grant: print specific recovery instructions ("run scripts/generate_tokens.py") and exit before opening Claude
- Cache successful checks for 6 hours to avoid latency on every launch

Impact: You discover the token issue at 9:01am, not at 9:43am after building a whole plan.

Fix #3 — Spreadsheet schema enforcement

Problem: A renamed tab, merged cell, or extra column silently corrupts the read.

Solution:

- First action of brdgit-campaign-reader: assert tab names exactly match the 5 expected names (no fuzzy match)
- Assert row 1 contains the exact column names from templates/spreadsheet-schema.md (case-sensitive)
- Reject merged cells: count rows and columns, refuse if any cell spans multiple cells
- Warn on extra columns (won't be read) and missing required columns (will fail)

Impact: Schema problems surface at Gate 0 with a clear error, not as wrong campaigns at Gate 4.

Architectural Fixes

Fix #4 — Move deterministic logic to Python scripts

Problem: LLM reasoning for budget math and API ordering is nondeterministic.

Solution:

- Move all numeric calculations (daily-to-monthly projection, batch totals, budget cap checks) to scripts/budget_math.py
- Move the API call sequence (10 ordered calls) into scripts/deploy_campaign.py — Claude orchestrates, but the sequence is hard-coded Python
- Skill brdgit-campaign-deployer becomes a thin wrapper that calls the Python script and reports results

Impact: Same input always produces same numbers; same plan always produces same API call order.

Fix #5 — Persist state between gates (P0)

Problem: Session crash between Gate 2 and Gate 3 wipes out approved copy.

Solution:

- After each gate approval, save full campaign-plan.json to /state/[campaign_name]/gate-[N]-approved.json
- Add a "Resume" prompt template: "Resume campaign [name] from last approved gate."
- Pipeline-runner checks /state/ on startup: if any in-flight campaign exists, offer to resume
- On final deploy success: archive /state/[campaign_name]/ to /reports/

Impact: Crashed sessions resume in 5 seconds. No re-approving copy you already approved.

Fix #6 — Signed approval log

Problem: No record of what state you approved vs what got deployed.

Solution:

- At each gate approval, write /reports/approvals/[campaign]-gate-[N]-[timestamp].json containing the full plan snapshot, gate number, approval string, and a SHA256 hash

- At deploy time, hash the plan again and refuse to deploy if it differs from the Gate 4 approved hash

Impact: Tamper-evident trail. If something goes wrong, you can prove exactly what was approved.

Fix #7 — Make creatives optional / async

Problem: Figma MCP timeout blocks deployment of search ads that don't need creatives.

Solution:

- In pipeline-runner, treat Gate 3 as optional for Search-only campaigns (RSA needs no creatives)
- For PMAX/Display: if Figma MCP fails, offer fallback: "Skip creatives for now and deploy with placeholders. Generate creatives separately later."
- Add prompt template: "Generate creatives only for [campaign] — campaign already deployed."

Impact: A flaky Figma session can't hold a Search campaign hostage.

Operational Fixes

Fix #8 — Conflict detection at Gate 1

Problem: Pipeline can create duplicate campaigns silently.

Solution:

- At Gate 1, query Google Ads MCP for existing campaign names matching the planned campaigns
- If a name collision exists: warn "Campaign X already exists in account (ID: Y, status: Z). Options: rename, update existing, or abort."
- Add explicit "update" path that queues UPDATE-mode API calls instead of CREATE

Fix #9 — Lock budget caps to env + checksum

Problem: Anyone with file access can raise the \$500/day cap.

Solution:

- Move daily_cap and monthly_warning to config/pipeline.yaml signed with a checksum stored in .env
- On startup, verify the checksum matches; if not, refuse to run and warn
- Raising the cap requires running scripts/unlock_budget.py with confirmation prompts and re-signing

Impact: A teammate with file access can't accidentally (or deliberately) bypass the cap.

Fix #10 — Real server-side dry run (P0)

Problem: "Dry run" only simulates Claude's reasoning, not Google Ads validation.

Solution:

- Use the Google Ads API `validateOnly: true` flag for every API call in the dry-run phase
- This sends the request to Google's servers, runs full policy + keyword + structure validation, and returns errors WITHOUT creating resources
- Show the validation results in Gate 4 ("✅ Google Ads accepted all 12 calls" or "❌ Headline 7 violates policy: [specific error]")

Impact: Policy disapprovals surface at Gate 4, not as a half-deployed campaign.

Fix #11 — Rate limit handling

Problem: Batch deploys of multiple campaigns hit Google Ads API quotas mid-run.

Solution:

- Wrap every API call in `scripts/deploy_campaign.py` with exponential backoff (retry 3x with 1s, 2s, 4s delays on `RESOURCE_EXHAUSTED` errors)
- In batch mode, deploy campaigns sequentially, not in parallel
- Add a 200ms sleep between API calls to stay under the per-second limit

Phase 2 Pre-emptive Fixes

Fix #12 — Per-industry compliance whitelist

Problem: Adapting copy to a regulated industry (healthcare, fintech) can hallucinate compliance claims.

Solution:

- Maintain `config/industry-claims.json` with allowed claims per industry: e.g., HealthTech allows ["HIPAA-compliant infrastructure"] but blocks ["FDA-approved", "clinically validated"]
- Before `brdgit-copy-adapter` outputs adapted copy, scan for any banned claim and refuse to emit
- Require legal sign-off before adding any new claim to the whitelist

Fix #13 — Statistical guards on auto-pause

Problem: Pausing keywords on 7-day data buries good keywords with low volume.

Solution:

- Require minimum 14 days of data AND minimum \$200 spend AND ≥ 100 clicks before any pause recommendation
- Use Bayesian conversion rate confidence intervals — pause only if 95% CI upper bound is below the campaign target CPL
- All pause recommendations still require human approval at the optimization gate

15. Phase 2 Roadmap — AI-Powered Optimization

Phase 2 extends the pipeline from one-shot campaign creation into ongoing, data-driven optimization. Not yet built.

15.1 Market Research-Driven Audience Expansion

Uses market research data and performance signals to identify new audience segments and industries to target — and automatically prepares the campaign briefs.

New skill: brdgit-market-research.md

```
EXPANSION OPPORTUNITIES IDENTIFIED
```

```
High Priority:
```

```
1. HealthTech (50-500 emp) — estimated CPL $90-120
```

```
Signal: 340 open "AI engineer" jobs in sector this month
```

```
Recommended angle: "HIPAA-compliant AI execution"
```

```
Campaign: BRDGIT_Search_HealthTech_FractionalAI_Q4
```

```
2. FinTech (50-200 emp) — estimated CPL $110-140
```

```
Signal: Series B/C companies without AI teams
```

```
Recommended angle: "Fraud detection + ML pipeline execution"
```

```
Generate campaign briefs for these? (yes / select / abort)
```

15.2 Cross-Industry Ad Messaging Adaptation

Takes winning ad copy from one industry and adapts it for a new target — preserving the structural angle, swapping vocabulary and pain points.

New skill: brdgit-copy-adapter.md (governed by Fix #12 whitelist)

ADAPTING: "Fractional AI" → Healthcare Audience

Original (B2B generic): H1: "AI Projects Keep Stalling?" (26)

Adapted (HealthTech): H1: "Patient AI Features Stalling?" (29) 

→ Same Pain Point angle, vertical vocab

Original: H1: "No AI Team? No Problem." (22)

Adapted: H1: "No ML Team? HIPAA-Ready." (24) 

→ Compliance angle whitelisted

Approve adapted copy? (approved / adjust / abort)

15.3 Creative Asset Adaptation by Industry

Extended skill: brdgit-creative-designer.md (Phase 2 extension)

- Copy overlay: per-industry headlines and CTAs
- Color emphasis: same brand palette, different primary/accent ratio per audience
- Imagery direction: abstract geometric (tech), clean (FinTech), warm + human (HealthTech)
- Frame naming includes industry code: BRDGIT_Q4_HealthTech_1200x628

15.4 Performance-Triggered Optimization Loop

New skill: brdgit-optimization-loop.md (governed by Fix #13 statistical guards)

WEEKLY OPTIMIZATION BRIEF — Week of [date]

PAUSE RECOMMENDED (passed 14d / \$200 / 100-click thresholds):

- [Keyword] — 87 clicks, \$0 conversions, \$430 spent
- [Keyword] — CTR 0.4%, Quality Score 3

ADD AS KEYWORDS (from search terms report):

- "ml engineer part time" → phrase, Fractional AI group
- "contract machine learning engineer" → exact, Brand group

ADD AS NEGATIVES:

- "ml engineer salary" (34 clicks, 0 conversions)
- "fractional cto" (wrong service)

BID ADJUSTMENTS:

- +20% bid for NYC metro (CPL 40% below average)
- -15% bid for mobile devices (0 conversions in 30 days)

Apply all? / Select which / Review each / Abort

15.5 Phase 2 Skill List (Planned)

Skill	Purpose	Status
brdgit-market-research	Identify new audiences and industries from data	Planned
brdgit-copy-adapter	Adapt winning copy to new audiences (whitelist-governed)	Planned
brdgit-optimization-loop	Weekly performance analysis with statistical guards	Planned
brdgit-competitor-monitor	Track competitor ads via ad library	Planned
brdgit-retargeting-builder	Launch retargeting campaigns from conversion data	Planned
brdgit-lookalike-builder	Create lookalikes from customer match	Planned

BRDGIT Ads Automation Pipeline — v1.1

Stack: Claude Code + Figma MCP + Google Ads MCP + Google Sheets MCP

No external orchestrators. No middleware. Terminal-native.